SNDA Game Protect Kit
SDK Manual
(Version3.0)

CONFIDENTIALITY/SECURITY WARNING

The information in the SDK is the property of Shanda Games Ltd. It must not be copied or distributed in any way without written permission from Shanda Games Ltd. The person allowed to peruse and keep the SDK shall be liable for losing, taking out and getting this document stolen.

Contents

Contents 2
1. GPK Introduction 4
1.1. GPK Function 4
1.2. GPK Features 4
1.2.1. Kernel technology 4
1.2.2. VM techonology 4
1.2.3. Dynamic Encryption and Decryption Technology 4
1.2.4. INTEMOR Technology 4
1.2.5. Hack-shielding Technology 5
1.3. GPK Advantages 5
1.3.1. Anti hacking 5
1.3.2. Anti Trojan 5
1.3.3. Security, Independence, Flexibility, Compatibility, Stability 5
1.3.4. Business Cooperation mode 5
2. GPK Implement 6
2.1. Steps 6
2.2. Implement Time 6
2.3. Anti hacking service after GPK integration 6
3. GPK SDK Files 7
3.1. Head File 7
3.2. LIB File 7
4. Development Guide of GPK Module Client SDK 8
4.1. Step One: Start GPK and create IGPKCltDynCode component 8
4.2. Step Two: Set dynamic encryption and decryption data 8
4.3. Step Three: Process Heartbeat Packet Buffer 9
4.4. Step Four: Release Component 9
5. Development Guide of GPK Module Server SDK 10
5.1. Step One: Create IGPKSvrDynCode Component 10
5.2. Step Two: Initialize dynamic encryption and decryption data 10
5.3. Step Three: Create IGPKCSAuthObject for each UserObject 10
5.4. Step Four: get the dynamic code and send it to the client 11
5.5. Step Five:Make Heartbeat Mechanism 11
5.6. Step Six: Check Heartbeat 11
5.7. Release IGPKCSAuth Component 13
5.8. Release IGPKSvrDynCode Component 13
5.9. Update the dynamic encryption and decryption database without restarting server 13
6. GPK Module Deployment and Work Flow 14
6.1. Upgrade Server Deployment 14
6.2. GS Deployment 14
6.3. Game Client Integration 14
6.4. Debugging programs in debugger 14

7. Reference to GPK SDK Interface 15
7.1. GPK Namespace 15
7.2. GPKStart Function 15
7.3. GPKCreateSvrDynCode Function 15
7.4. IGPKSvrDynCode Component 15
7.4.1. Introduction 15
7.4.2. IGPKSvrDynCode::LoadBinary 16
7.4.3. IGPKSvrDynCode:: Release 16
7.4.4. IGPKSvrDynCode:: AllocAuthObject 16
7.5. IGPKCltDynCode component 16
7.5.1. Introduction 16
7.5.2. IGPKCltDynCode::SetDynCode 16
7.5.3. IGPKCltDynCode:: Release 17
7.5.4. IGPKCltDynCode:: ProcessAuth 17
7.6. IGPKCSAuth component 17
7.6.1. Introduction 17
7.6.2. IGPKCSAuth::GetAuthData 18
7.6.3. IGPKCSAuth::CheckAuthReply 18
7.6.4. IGPKCSAuth:: GetCSAuthDynCode 18
7.6.5. IGPKCSAuth:: Release() 19

1. GPK Introduction

1.1. GPK Function

GPK is a kind of ring0 game protection tools. The function is as follow:

1、Anti debug and all kinds of hacking tools, like Ollydbg, Rootkit.

2、Prevent controls from Macro & Auto-Mouse.

3、Protect game process from memory hacking.

4、Make a HASH check in the memory segment appointed by the game procedure with the anti-modifying result.

5、Make dynamic encryption and decryption algorithm for game packet. The game is able to adopt the GPK algorithm directly without redesign by yourself. However, it is possible for you to design your own algorithm plus the GPK algorithm.

6、Anti speed hack.

7、Anti Key-loggers of Ring0 and Ring3 to efficiently protect the accounts.

8、Anti Trojan with no Trojan injection or modifying in game process.

9、Upgrade the dynamic algorithm and anti hacking module without restarting the server to kill the new hacking much faster.

1.2. GPK Features

1.2.1. Kernel technology

GPK adopts all kinds of kernel technology to stop the hacking and give a real-time protection to all key points.

1.2.2. VM techonology

GPK makes full use of the mature and advanced VM technology without increasing the consumption of system resources. It enhances the reverse difficulties in making hacking programs and Trojan and the hackers unable to reverse or able to do so with high cost.

1.2.3. Dynamic Encryption and Decryption Technology

GPK uses dynamic encryption and decryption to enhance the reverse difficulties in making hacking programs.

1.2.4. INTEMOR Technology

By means of all kinds of the hacking characteristics, GPK will automatically identify them and feedback to the server and take measures according to the needs of the cooperative side.

1.2.5. Hack-shielding Technology

GPK will protect the key data in the memory and monitor the sensitive codes in order to make it impossible for the hacking programs to modify the in-game data or codes.

1.3. GPK Advantages

1.3.1. Anti hacking

Anti hacking is the most basic function of GPK, which is originally designed to make full preparation for many kinds of hacking. In terms of the GPK architecture, it has put an end to the possibilities of the non-client bot, while it attacks all kinds of hacking programs with tens of advanced technologies.

1.3.2. Anti Trojan

GPK makes the game immune to most of the recent Trojan technologies.

1.3.3. Security, Independence, Flexibility, Compatibility, Stability

GPK's anti hacking system boasts the powerful security mechanism with continuous upgrade.

GPK adopts the whole or part dynamic upgrade skills and its control is independent of the game itself. The game upgrade can go without shutdown when GPK is upgrading.

For the high flexibility of GPK, various kinds of function is able to realize by module with convenient switch and extension. Such flexibility can also complete the operation of other games' running for the cooperative side.

GPK anti hacking OS client is compatible with all windows OS from Win2000. Its server is compatible with all common server OS. Up to now, there is no compatible problems from its several hundred thousand of users.

GPK owns strong stability.

1.3.4. Business Cooperation mode

On the condition of assuring the mutual interest, GPK supports all kinds of commercial cooperation mode to get a win-win result. GPK is able to make a special version to meet the needs of the client.

2. GPK Implement

2.1. Steps

1、Service contact.

2、Provide GPK module SDK for the game developer.

3、The developer integrates GPK module into the game server.

4、Server configuration

5、Test.

6、Complete.

2.2. Implement Time

About 7 to 10 days (including test, without service configuration).

2.3. Anti hacking service after GPK integration

The appearance of GPK is for the better anti hacking. Although GPK's active defense mode is immune to many cheating software, it is possible for some hacking ways which are special for the game development to avoid the anti hacking defense. Therefore, it is necessary to form the quick and flexible responding system for such kind of hacking. So we will keep on the after service for those games integrated with GPK.

When the game providers find the new kind hacking, they can send the hacking samples to our service mailbox gpk@snda.com with the relevant game account attachment. If it is charge hacking, pls give the hacking account as much as possible for the quicker solution. After receiving the mail, we will respond within 2 hours. For the upgrading hacking, we will complete the block update within 2 working days, while for the brand-new hacking, within 4 working days. For the few emergent cases, we can finish the update in one day in priority.

3. GPK SDK Files

GPK will provide some head files, lib and DLL files to the game developers. All the files and their position in the GPK modules we provide is are as follow:

3.1. Head File

They are in ‘Include’ directory.

Name	Description
GPKitClt.h	It is client head file, in ‘Include’ directory and is used to the integration of the client.
GPKitSvr.h	It is server head file, in ‘Include’ directory and is used to the integration of the server.

3.2. LIB File

LIB file is in ‘LIB’ directory.

Name	Description
GPKitClt.lib	It is client lib, in 'LIB' directory and is used to integration of the Release Game Client.
GPKitSvr.lib	It is server lib, in 'LIB' directory and is used to integration of the Release Game Server.
GPKitSvrD.lib	It is server lib, in 'LIB' directory and is used to integration of the Debug Game Server.

4. Development Guide of GPK Module Client SDK

4.1. Step One: Start GPK and create IGPKCltDynCode component

Entry function of main procedure should be called before the game window message loop is established to prevent the hack from using HOOK before starting GPK. For example, the constructor of WinMain or MFC's CxxxApp will call SDK interface function GPKStart to start protection and return the pointer of IGPKCltDynCode interface.

Example code：

```
#include “GPKitClt.h”     //GPK head file
using namespace SGPK    //GPK namespace
……
IGPKCltDynCode *pCltDynCode =
GPKStart(“http:// localhost/GPK”, “Game”);
if(NULL == pCltDynCode)
{
        printf("GPKStart failed\n");
}
…….
////create game window
```

4.2. Step Two: Set dynamic encryption and decryption data

Connect to the GS and get the dynamic code data, then set GPK dynamic code data. The client will use IGPKCltDynCode to decode only after finishing dynamic code data setting. Call IGPKCltDynCode::SetDynCode to set dynamic encryption and decryption data.

Example code:

```
//save the dynamic code data from server, dynamic code size<12K
unsigned char* lpDynCode[14000]    ;
int nlen;    //Receive data length.
nlen = ClientSock.Recv(lpData, 14000);//receive data from GS, length is nLen
pCltDynCode->SetDynCode(lpDynCode, nlen);
//continue other code of game client;
```

4.3. Step Three: Process Heartbeat Packet Buffer

Call IGPKCltDynCode::ProcessAuth to process heartbeat packet buffer,and send it to the server. Notice: there is no change in the length because the function will do cover the original data!

Example code:

```
unsigned char lpData[1024]    ; //save the data from the server
int nlen;    //receive the data length
nlen = ClientSock.Recv(lpData,1024);
pCltDynCode->ProcessAuth(NULL,NULL,lpData,nlen);// Process Heartbeat Packet
ClientSock.Send(pData,len); //send to the server
```

4.4. Step Four: Release Component

When the game client exits, call IGPKCltDynCode::Release;

Example code:

```
pCltDynCode->Release();
```

5. Development Guide of GPK Module Server SDK

5.1. Step One: Create IGPKSvrDynCode Component

After server procedure starts and before Socket starts, call GPKCreateSvrDynCode to create IGPKSvrDynCode component.

Example code：

```
#include “GPKitSvr.h”      //GPK server head file
using namespace SGPK    //GPK namespace


IGPKSvrDynCode * pSvrDynCode = GPKCreateSvrDynCode();
if(NULL == pSvrDynCode)
{
        printf("Create SvrDynCode component failed\n");
}
```

5.2. Step Two: Initialize dynamic encryption and decryption data

Create IGPKSvrDynCode component and call IGPKSvrDynCode::LoadBinary to initial dynamic encryption and decryption database.

Example code:

```
int nBinCount;
//load dynamic code database
nBinCount = pSvrDynCode->LoadBinary(“DynCode\\Server”, “DynCode\\Client”);
if(0 == nBinCount || -1 == nBinCount);      //error
    goto fail_ret;
```

5.3. Step Three: Create IGPKCSAuthObject for each UserObject

When each client connects to the server, call IGPKSvrDynCode::AllocAuthObject to Create IGPKCSAuthObject.

Example code:

```
Typedef struct _UserObject
{
      SGPK::IGPKCSAuth *p_gpkAuthObject;
```

```
…// Other definitions of Game itself
} UserObject;
…
UserObject->p_ gpkAuthObject = pSvrDynCode->AllocAuthObject();
```

5.4. Step Four: get the dynamic code and send it to the client

After Create the IGPKCSAuthObject, call IGPKCSAuth::GetCSAuthDynCode to get the dynamic code Buffer and send it to client.

Example code:

```
const unsigned char *pCode = NULL;
int nLen = UserObject->p_ gpkAuthObject-> GetCSAuthDynCode (&pCode);
ClientSock.Send(pCode,nLen);//send to client
```

5.5. Step Five:Make Heartbeat Mechanism

For each UserObject online,we should call IGPKCSAuth::GetAuthData to make different Heartbeat Buffer by 1~2 minutes,and send it to the client immediately.if the call return value is 0,it mean's find hack action,game should log the user's info and kick the user.

Example code:

```
const unsigned char *p = NULL;
int len = UserObject->p_ gpkAuthObject->GetAuthData(&p,NULL,NULL);
if( len != 0 )
{
    ClientSock.Send(pData,len); //send to client
}
Else
{
    Log(UserObject);
    CloseSocket(UserObject);
}
```

5.6. Step Six: Check Heartbeat

Onrecv GPK heartbeat packet,wo should call IGPKCSAuth::CheckAuthReply to check the Heartbeat info. if the call return value isn't CHECK_SUCCEED,it mean's find

hack action,game should log the user's info,log CheckAuthReply's return value and kick the user.

Example code:：

```
void CServerSocket::OnReceive(int nErrorCode)
{
   unsigned char buff[512];
   int len = Receive(buff,512);
   const unsigned char *p = NULL;
   …
   int ret = UserObject->p_ gpkAuthObject->CheckAuthReply(&p,buff,len);
   if( ret != CHECK_SUCCEED )
   {
      If( ret == CHECK_FIND_CHEAT_NEED_KICK )
      {
            PCHEAT_INFO CheatInfo = (PCHEAT_INFO)p;
            Log(UserObject,ret,CheatInfo);
            CloseSocket(UserObject);
      }
      else if( ret == CHECK_FIND_CHEAT_ONLY_LOG )
      {
            PCHEAT_INFO CheatInfo = (PCHEAT_INFO)p;
            Log(UserObject,ret,CheatInfo);
      }
      else
      {
            Log(UserObject,ret);
            CloseSocket(UserObject);
      }
   }
   …
}
```

5.7. Release IGPKCSAuth Component

When user logged out of game, call IGPKCSAuth::Release to release IGPKCSAuth component.

Example code:：

UserObject->p_ gpkAuthObject->Release();

5.8. Release IGPKSvrDynCode Component

When exiting the game server process, call IGPKSvrDynCode::Release to release IGPKSvrDynCode component.

Example code:：

pSvrDynCode->Release();

5.9. Update the dynamic encryption and decryption database without restarting server

If the new hacking appears, there is no need to restart the server while the dynamic encryption and decryption database maybe need upgrading, therefore users is able to get better game experience.

At first, the server should have a correct configuration.

Back up the original bin file.

Set the new bin file to the relative directory, like DynCodeBin\Clien\ and DynCode\Server\.

Add a GM command in the server procedure, like RelaodDynCode (can be self-defined). The format is as follows:

ReloadDynCode

The server procedure receives the command to call the LoadBinary in IGPKSvrDynCode and reload a new bin file.

Example:

Input GM command:ReloadDynCode,Server calls the LoadBinary in IGPKSvrDynCode.

nBinCount = pSvrDynCode->LoadBinary(“DynCode\\Server”, “DynCode\\Client”);

if(0 == nBinCount || -1 == nBinCount); //error

goto fail_ret;

reload the dynamic code pair in the two directories. Since then, new users will adopt new dynamic encryption and decryption database.

6. GPK Module Deployment and Work Flow

6.1. Upgrade Server Deployment

Upload all the files in the directory of Targetdir including the GPK.S file to the update server.

6.2. GS Deployment

1、 Copy GPKitSvr.dlll(Release Version) or GPKitSvrD.dll(Debug Version) to the directory of the server program.

2、 In the server create a directory for dynamic code like DynCodeBin which can be self-defined and must be the same to the call in SDK, and two subdirectories like server and client for the saving of server and client dynamic code files with the file extension name .bin, like 01.bin or 02.bin. Each file in Server0 should have a file with the same name in the client0. The two files with the same name form a pair of dynamic code. The two files with the same name have different content and should be provided by the anti-hacking group. If you want to update the dynamic code without restarting, more subdirectories are to be created, like server1 and client1. Assign the pairs of dynamic codes to these subdirectories. **In Redist\Server, we have prepared a directory DynCodeBin and the related files which can be copied to the directory of the server.**

6.3. Game Client Integration

1、 Copy the files in GPK subdirectory under Redist\Client in SDK packet(seven in all) to the directory of the client and then GPKitClt.dll(Release Version) or GPKitCltD.dll(Debug version, use GPKitClt.dll on the condition of release) to the client directory. For example, a game client directory is C:\GameOne, then add a GPK directory and a GPKitClt.dll file in it. GPK folder includes seven files, AutoUpdate.dll is a must.

2、 Start the protection through GPK SDK interface (refer to the forth part: Development Guide of GPK Module Client SDK).

6.4. Debugging programs in debugger

Since the powerful anti-debug function of GPK, the game procedure run in the dubugger is unable to be interrupted after the integrity of GPK. Therefore, it brings some trouble in the debug during the game development. Please add a macro for the relevant GPK code on both client and server to have control. When compiling the debug version, it can close GPK code through the macro.

7. Reference to GPK SDK Interface

7.1. GPK Namespace

GPK module namespace is SGPK

7.2. GPKStart Function

Function：

Automatically updating GPK components, start the kernel protection and create IGPKSvrDynCode components.

Function declaration：

IGPKCltDynCode* GPK_API GPKStart(IN LPCSTR UpdateServerURL,

IN LPCSTR GameCode);

Parameter：

UpdateServer：[in] update server URL; no update if the parameter is null（The case of NULL is mainly for the debug or test）.

GameCode: [in] game label, length is 8 bytes, if plus \0, the length is 9 bytes at most.

Return value：

If success, return the pointer of the IGPKCltDynCode interface，otherwise return NULL.

7.3. GPKCreateSvrDynCode Function

Function：

Create IGPKSvrDynCode

Function declaration：

IGPKSvrDynCode * GPK_API **GPKCreateSvrDynCode**()。

Parameter：

NULL。

Return value：

If success, return the pointer of the IGPKSvrDynCode interface，otherwise return NULL.

7.4. IGPKSvrDynCode Component

7.4.1. Introduction

IGPKSvrDynCode is the server component, mainly used to the dynamic encryption and decryption, and update dynamic algorithm database without server reset.

7.4.2. IGPKSvrDynCode::LoadBinary

Function:

Load dynamic code file pair

Function declaration:

int GPK_API **LoadBinary**(const char * pszSvrBinDir, const char * pszsCltBinDir)

Parameter:

pszSvrBinDir ： [in] Directory of the server dynamic code files

PszsCltBinDir：[in] Directory of the client dynamic code files

Return value:

If success, return the number of the loading dynamic code files, otherwise return -1.

7.4.3. IGPKSvrDynCode:: Release

Function:

Release IGPKSvrDynCode component.

Function declaration:

void GPK_API Release()

Parameter:

None

Return value:

Void

7.4.4. IGPKSvrDynCode:: AllocAuthObject

Function:

Create IGPKCSAuth object for session connection.

Function declaration:

IGPKCSAuth* GPK_API AllocAuthObject()

Parameter:

None

Return value:

If success, return the pointer of the IGPKCSAuth Object, otherwise return NULL.

7.5. IGPKCltDynCode component

7.5.1. Introduction

IGPKCltDynCode components are tailored to the client, mainly used to the encryption and decryption of the dynamic algorithm to protect the integrity of the GPK kernel module.

7.5.2. IGPKCltDynCode::SetDynCode

Function:

Set sndc dynamic code data. It should be called before the first time use of the

dynamic code encryption and decryption.

Function declaration：

bool GPK_API SetDynCode(const unsigned char* lpDynCode, int nLen)

Parameter：

lpDynCode：[in] Receive the sndc dynamic data from the server.

nLen： [in] Length of the sndc dynamic data

Return value：

If success, return true, otherwise return false.

7.5.3. IGPKCltDynCode:: Release

Function：

Release IGPKCltDynCode components.

Function declaration：

void GPK_API Release()

Parameter：

None

Return value：

void

7.5.4. IGPKCltDynCode:: ProcessAuth

Function：

Process Auth Data From Server.

Function declaration：

Int GPK_API ProcessAuth(void * Object, fnGPK_CltCallBack SendToSvr,
unsigned char * lpData, unsigned long nLen)

Parameter：

Object:Object Pointer for CallBack，can be socket or user's class,etc,also can be NULL. Specific Object Type is decided by Game Framework.

SendToClt: CallBack Function.

lpData：[in,out]Data pointer for Processed Data.

nLen：Length of Data.

Return value：

If success, return the length of data, otherwise return 0.

7.6. IGPKCSAuth component

7.6.1. Introduction

IGPKCSAuth Component is Server Component for detecting malware software and the status of GPK.

7.6.2. IGPKCSAuth::GetAuthData

Function：

Generate CSAuth Data

Function declaration：

Int GetAuthData(const unsigned char **ppDataRet,void *Object,fnGPK_SvrCallBack SendToClt=NULL)

Parameter：

ppDataRet:Data Pointer

Object: Object Pointer for CallBack，can be socket or user's class,etc,also can be NULL. Specific Object Type is decided by Game Framework.

SendToClt: CallBack Function

Return value：

If success, return the length of data, otherwise if return -1 means exception,if return 0 means no response in last CheckAuthReply called.Maybe bot blocked the Auth Package.

7.6.3. IGPKCSAuth::CheckAuthReply

Function：

Check the Auth Data from Client.

Function declaration：

int CheckAuthReply(const unsigned char **ppDataRet,unsigned char * lpData, unsigned long nLen)

Parameter：

ppDataRet:CSAuth Data Pointer

lpData:Data Pointer for output.

nLen:Length of Data.

Return value：

Return value reference to GPK_CHECK_INFO

7.6.4. IGPKCSAuth:: GetCSAuthDynCode

Function：

Get the dynamic code data ready to send to the client.

Function declaration：

int GetCSAuthDynCode(const unsigned char **ppCodeRet)

Parameter：

ppCodeRet: [out] Output the pointer ip which generates in the function and save the dynamic code data. Set *ppCodeRet as NULL beforehand.

Return value：

If success, return the length of dynamic code data, otherwise return 0.

7.6.5. IGPKCSAuth:: Release()

Function：

Release IGPKCSAuth Component

Function declaration：

void GPK_API Release()

Parameter：

None

Return value：

Void